Integer multiplication below $n \log n$

An illustrated reading edition

Original manuscript: OpenAI

Added explanations and figures prepared with OpenAI Codex

Second illustrated edition, October 12, 2026

Original manuscript dated September 23, 2026

Abstract

A guided explanation of the original argument, with 24 explanatory figures, worked arithmetic examples, a map of the recursive construction, and explicit distinctions between source theorem claims, illustrative calculations, and conditional proof interfaces. The exposition begins with digit convolution and builds toward frame geometry, tape recurrences, transforms, and exact recovery. It is not a new proof or an official revision of the source paper.

Contents

Reader guide: Start with the problem, not the symbols **3**

Reader guide: The question, the claim, and how to read it **4**

Reader guide: See exactly which digit products get added **6**

Reader guide: From long multiplication to convolution **7**

Reader guide: A matrix transpose is a physical rearrangement **8**

Reader guide: Why moving data is part of the mathematics **10**

Reader guide: Vectors, rank, and a frame you can draw **11**

Reader guide: Borrowed workspace and changing representations **12**

Reader guide: How unwanted sources cancel out **15**

Reader guide: Why more calls can still mean less work **16**

Reader guide: How a finite rank saving becomes an exponent **17**

Reader guide: What a transform actually changes **18**

Reader guide: The smallest transform step, applied on many axes **20**

Reader guide: Making roots of unity into signed shifts **21**

Reader guide: Changing the grid without sorting everything **22**

Reader guide: Small error must stay small after rescaling **24**

Reader guide: Why improving one part may change nothing **24**

Reader guide: Putting exactness and the running time together **25**

Reader guide: What a successful check actually tells us **27**

Reader guide: What follows, and what still needs checking **29**

Reader guide: Notation and source map **30**

Start with the problem, not the symbols

Added explanation First pass: ordinary arithmetic is enough for this opening.

You have two long strings of binary digits. You want their exact product. The research question is how the required work grows when the strings become longer. It is not a question about whether multiplication is possible, and it is not a promise that a new program will beat your calculator tomorrow.

Think of three bills. There is a bill for combining numbers, a bill for moving the right pieces next to one another, and a bill for making sure the final answer is exact. This paper proposes a way to reduce the first two bills together while still paying the third. The illustrations below show actual data paths: the left picture changes where records are stored; the right picture connects pairs that are combined at successive transform levels.

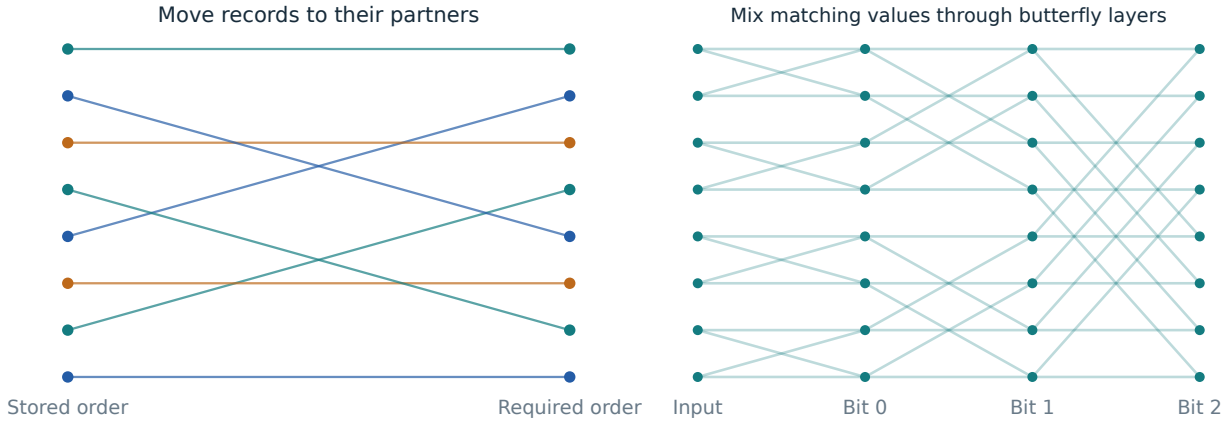


Figure E1. Two distinct jobs inside fast multiplication. Left: records must be brought into a useful order. Right: an eight-entry butterfly network combines entries differing in one address bit per level. These are small structural illustrations, not the paper's enormous circuit.

What is a bit? A bit is a zero or a one. In decimal, the places are ones, tens, hundreds. In binary, they are ones, twos, fours, eights. Thus binary 1011 means $1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 11$. This input has four bits. The paper writes n for the bit length, not the number's value. Leading zeroes are allowed: 0011 is also a four-bit input, even though its value is three.

What does a logarithm count? For powers of two, $\log_2 n$ counts how many doublings take one to n. Since $1, 2, 4, 8, 16$ takes four doublings, $\log_2 16 = 4$. The distinction between n and $\log_2 n$ is crucial. When a bit string doubles in length, n doubles but $\log_2 n$ increases by just one.

What does a running-time formula mean? Schoolbook multiplication pairs each of roughly n digits with roughly n others, suggesting n^2 elementary digit products. More sophisticated algorithms avoid doing all those pairs separately. A bound such as $O(n \log n)$ says that, beyond a threshold, the work is at most a fixed constant times $n \log n$. It does not specify the constant, the threshold, or the number of seconds on a particular computer.

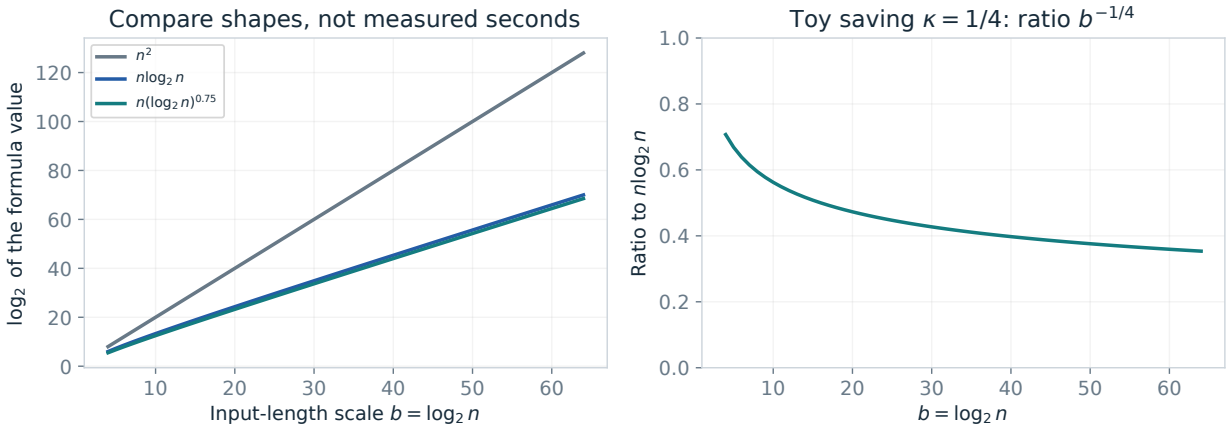


Figure E2. Growth rates need the right scale. Left: the vertical axis is the logarithm of a formula's value, so exponentially different values fit on one plot. Right: dividing the toy improved formula by $n \log_2 n$ exposes its smaller growth. The saving $1/4$ is deliberately large enough to see; the original paper uses 2^{-182}. Neither panel is a measurement of a multiplication program.

Two ways to read this edition. For a first pass, follow the worked examples, pictures, and "What to carry forward" paragraphs. They build the vocabulary before the technical argument uses it. For a second pass, read the equations and then the corresponding original proofs in the expanded paper. Understanding why the pieces fit is a different goal from checking every theorem. The guide supports the first goal and provides a route toward the second; it does not turn the full proof into elementary arithmetic.

A quick check. If an input grows from 16 bits to 32 bits, n doubles and $\log_2 n$ grows from four to five. The value of the input integer can be vastly larger than 32. Keeping these three quantities separate prevents many later confusions.

What to carry forward. The paper tries to save a little work at an enormous number of scales. To understand it, follow both the arithmetic and the movement of data.

The question, the claim, and how to read it

Added explanation Start here; no Fourier analysis is assumed.

Multiplication looks finished when we learn it at school. But the number of elementary steps needed to multiply two extremely long binary strings is a different question. The input length is the number of bits, not the numerical value of the input. A number with a million bits is one input of length a million; the fact that its value may be near $2^{1,000,000}$ does not change that length.

The manuscript studied here is OpenAI's *Integer multiplication below $n \log n$*, dated September 23, 2026, at source commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`. It states a worst-case bound

$$T(n) = O(n(\lg n)^{1-\kappa}), \qquad \kappa = 2^{-182}.$$

Here $\lg n = \max\{\lceil \log_2 n \rceil, 1\}$. The machine must output the exact product for every input, including inputs with leading zeroes. The machine model uses a fixed finite alphabet and a fixed finite number of one-dimensional tapes. Reading, writing, and moving data all cost time.

What this edition does and does not establish. The original text states its result as a theorem. This edition explains that argument; it does not independently establish its validity. The community repository treats its improvements as conditional on the original framework and retained implementation and analytic interfaces. That distinction matters: checking an exact finite network or a list of rational inequalities does not by itself prove that a single tape machine implements the complete algorithm within the asserted bound for all input sizes. Original theorem statements are retained as source text in the full edition, not recast as new findings by this guide. This is an independently prepared explanatory edition, not an official OpenAI revision or endorsement.

Why a tiny positive exponent matters. Ignoring constants only for this comparison, divide the proposed bound by $n \log_2 n$. The quotient is $(\log_2 n)^{-\kappa}$, which tends to zero for every fixed $\kappa > 0$. This is an asymptotic distinction, however tiny κ is. It is not a prediction of a useful speedup on a computer. Even reducing this idealized ratio to $1/2$ requires

$$(\log_2 n)^{\kappa} = 2, \qquad \log_2 n = 2^{1/\kappa}, \qquad n = 2^{\,2^{1/\kappa}}.$$

With $\kappa = 2^{-182}$ this is an extraordinary scale. Unknown constants, thresholds, and the actual implementation make a practical comparison still more demanding. A larger κ means a better exponent saving, not an equal percentage improvement in measured runtime.

The argument in one paragraph. Turn digit multiplication into convolution. Use transforms to turn convolution into smaller pointwise products. Make two expensive ingredients cheaper: rearranging the addresses of stored data, and applying many two-point transform steps. Both improvements come from finite linear networks whose intermediate representations are chosen carefully. Prove a recursive saving after charging every auxiliary role and every representation change. Then show that layouts, resampling, precision, and exact recovery fit under the same final bound.

A route through the expanded paper. The new foundation chapters introduce each prerequisite with a concrete example before its technical use. First read the digit example and the tape explanation. Next read restoration, common frames, and the recurrence: these explain where the proposed saving comes from. The butterfly and polynomial examples explain what consumes the saving. The final chapters explain why reordering and rounding cannot be afterthoughts. In the full edition, each added explanation precedes the original technical material it illuminates. Added figures use the prefix E; original section, equation, theorem, and figure numbering are preserved.

What to carry forward. The central question is whether the entire computation, including data movement and exact recovery, can save a positive power of $\log n$.

Source and attribution. The original paper and its bibliography are preserved under the Apache-2.0 license in the accompanying source package. Its historical discussion credits Schönhage–Strassen, Harvey–van der Hoeven, Nussbaumer and Quandalle, and other prior work. The subsequent CrocSwap repository preserves community contributions separately. New explanatory prose, figures, and example checks in this edition were prepared with OpenAI Codex assistance on October 12, 2026. The source snapshot is frozen; this document is not a live record leaderboard.

See exactly which digit products get added

Added explanation Convolution without new machinery.

Every term in long multiplication has a destination place. In 23×14, the 3×4 product contributes to the ones place. Both 3×1 and 2×4 contribute to the tens place, because one factor in each product is a tens digit. Finally 2×1 contributes to the hundreds place. The diagonal colors in the grid identify those destinations.

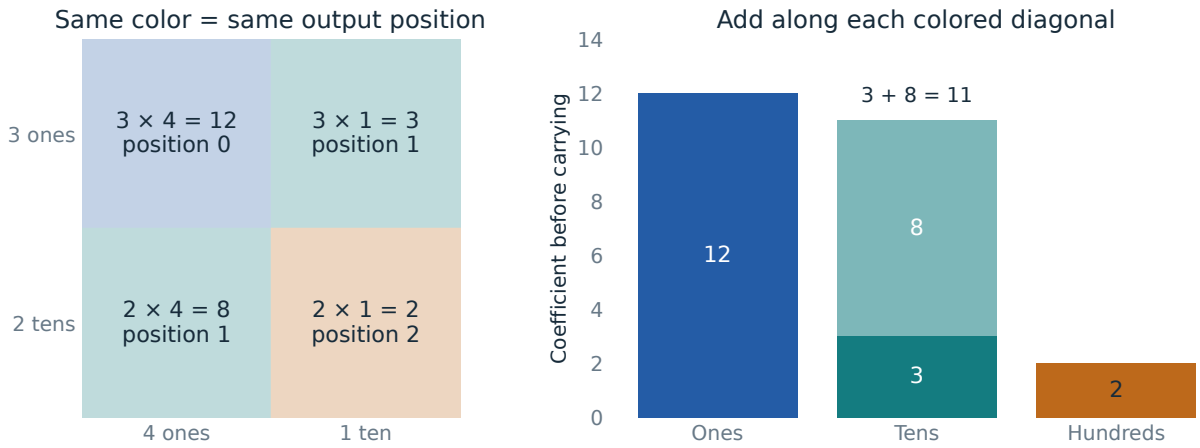


Figure E3. Convolution is addition along equal-place diagonals. The two teal cells contribute 3 and 8 to the same coefficient, giving 11. The coefficient list is $[12, 11, 2]$, before carrying. A cell is a specific digit product, not a generic stage of an algorithm.

Number digit positions starting at zero. A digit in position i multiplies a digit in position j into position $i + j$, because $10^i 10^j = 10^{i+j}$. That is the whole reason for the rule "add products with the same position sum." The word *convolution* names this rule. It is the same rule in binary or any other radix.

Why change to a polynomial? Replace the radix ten by a variable x. The two inputs become $A(x) = 3 + 2x$ and $B(x) = 4 + x$. Their product is a quadratic: it can be written $C(x) = c_0 + c_1 x + c_2 x^2$ for three unknown coefficients. At $x = 10$ it must give the integer product. But ten is not the only useful place to evaluate it. Easier sample points can reveal its coefficients.

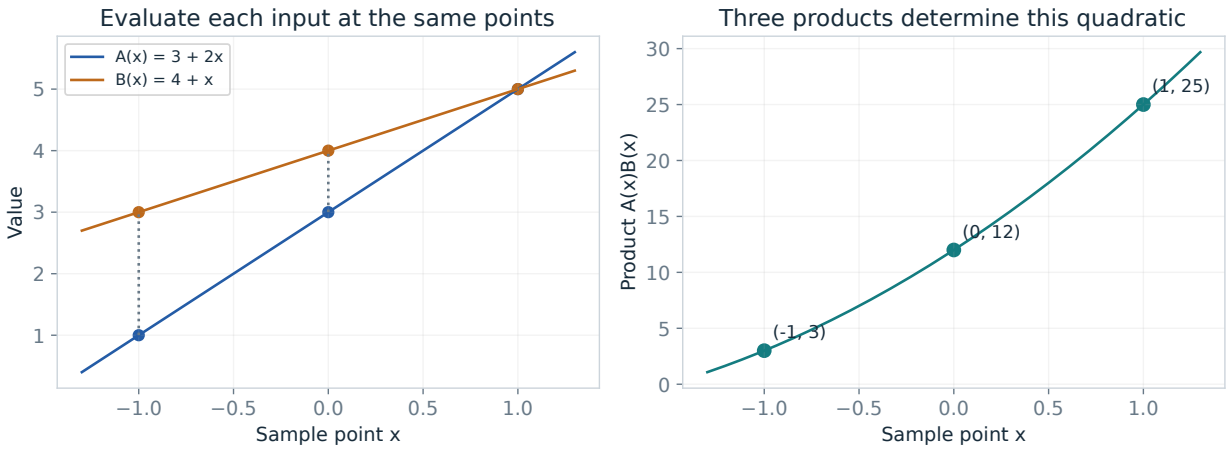


Figure E4. Compute values first, then recover coefficients. At $x = -1, 0, 1$, the input values multiply to $3, 12, 25$. Exactly one polynomial of degree at most two passes through those three distinct sample points. The curves are these specific polynomials, not a metaphor for a transform.

At zero, $C(0) = c_0 = 3 \times 4 = 12$. At one, $C(1) = 5 \times 5 = 25$. At minus one, $C(-1) = 1 \times 3 = 3$. Thus

$$c_0 = 12, \qquad c_1 = \frac{25 - 3}{2} = 11, \qquad c_2 = \frac{25 + 3}{2} - 12 = 2.$$

Subtracting the last two sampled equations isolates the odd-power term; adding them isolates the even-power terms. Evaluating at ten now gives $12 + 11 \cdot 10 + 2 \cdot 100 = 322$.

Where this example stops. This tiny interpolation example is not the paper's fast algorithm. It shows why "evaluate, multiply matching values, reconstruct" can work at all. For huge inputs, evaluation and reconstruction must themselves be fast and well controlled. Fourier transforms choose highly structured sample points so that those steps can share work. The next foundation chapter explains what their patterns look like.

What to carry forward. Convolution groups equal-place products. A transform provides another way to compute the same coefficients; it does not change the product.

From long multiplication to convolution

Added explanation A complete small example.

Write an integer in radix B as $a_0 + a_1 B + \cdots + a_{q-1} B^{q-1}$. The digits are listed from least significant to most significant. For the explanation take $B = 10$; the paper uses a power of two so that digit splitting is a binary operation. Introduce a formal variable x and form $A(x) = \sum_i a_i x^i$. Then the original integer is exactly $A(B)$.

For two inputs, the coefficients of the polynomial product satisfy

$$A(x)B(x) = \sum_k c_k x^k, \qquad c_k = \sum_{i+j=k} a_i b_j.$$

The second input polynomial is denoted $B(x)$ here; the unparenthesized B still denotes the radix. The sum defining c_k is *convolution*. It groups digit products that have the same place value. Unlike digits, the coefficients c_k can exceed the radix. Carry propagation repairs that after the coefficients have been computed.

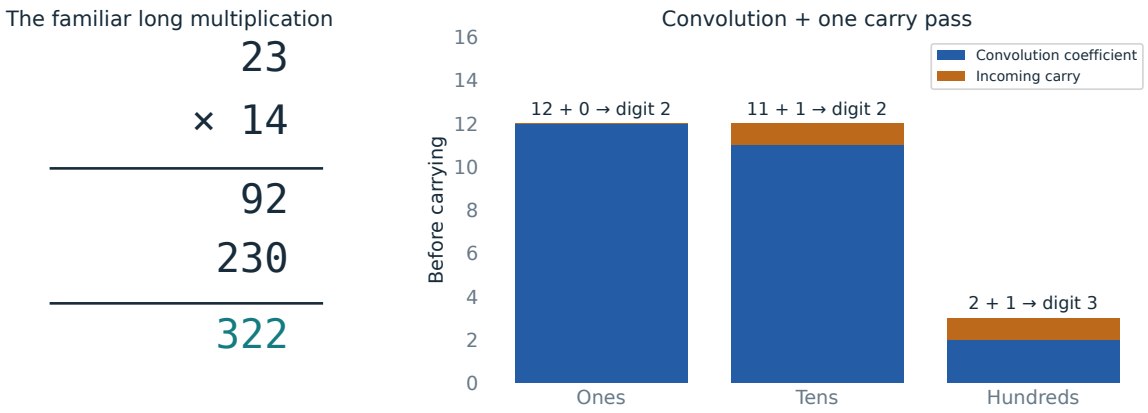


Figure E5. One multiplication, three representations. The coefficient list $[12, 11, 2]$ belongs to powers x^0, x^1, x^2. The carry is passed from lower to higher positions. The result is 322.

For $23 \cdot 14$, the coefficient of x is $3 \cdot 1 + 2 \cdot 4 = 11$. At position zero, 12 writes digit 2 and carries 1. Position one receives $11 + 1 = 12$, again writing 2 and carrying 1. Position two receives $2 + 1 = 3$. This is ordinary multiplication reorganized, with no approximation so far.

Why transforms enter. For a suitable transform F, convolution can be recovered from products of corresponding transform entries. Evaluating two polynomials at many roots of unity, multiplying those values, and transforming back replaces many cross-products by structured additions and pointwise products. The benefit depends on computing the transforms and arranging their inputs cheaply. The paper's transforms are normalized, so known scale factors must be put back; the slogan "transform, multiply, inverse transform" hides those factors.

Why padding is necessary. A cyclic convolution treats positions modulo a length S. A contribution at position S then wraps to position zero. That is useful algebraically but would corrupt this integer product. For degree at most $2q - 2$, choosing $S > 2q - 2$ and padding unused entries with zero prevents nonzero wraparound. The original assembly proves its chosen prime-axis product has enough capacity. It also bounds each coefficient before choosing its storage precision.

What to carry forward. The hard task is now to compute all convolution coefficients cheaply and accurately enough to recover the integers, then carry once.

Original source anchors: `00-introduction.tex; 08-assembly.tex, eq:capacity`.

A matrix transpose is a physical rearrangement

Added explanation Follow all nine records.

Read the 3×3 grid row by row: $0, 1, 2, 3, 4, 5, 6, 7, 8$. Now read it column by column: $0, 3, 6, 1, 4, 7, 2, 5, 8$. The records have not changed internally. Their positions in the stream have. The crossing paths show the exact permutation, including the records whose positions stay fixed.

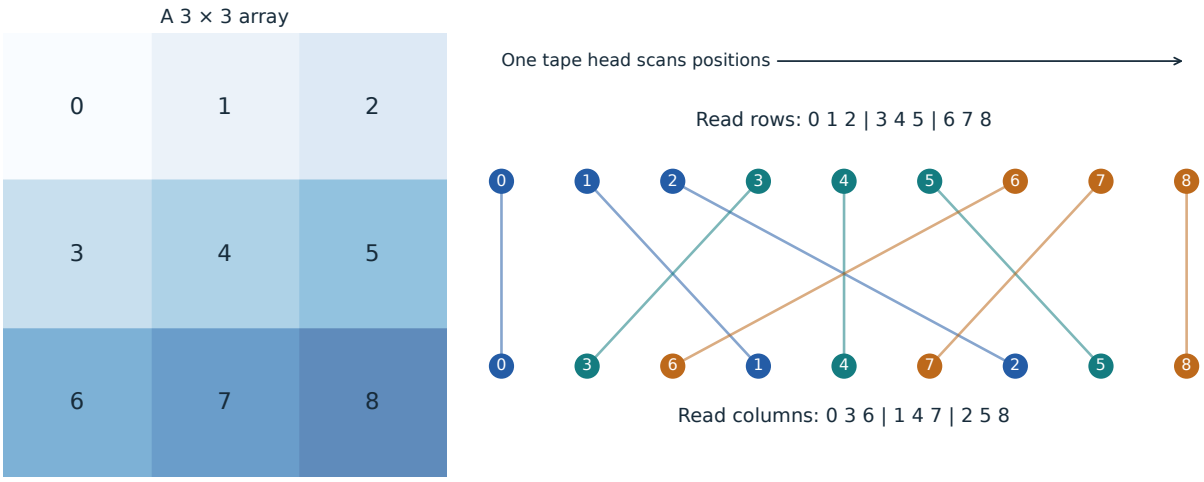


Figure E6. The geometry of one concrete data move. Colors identify original rows; every numbered record is connected to its destination. The top scan arrow represents a tape head moving through consecutive positions. It cannot teleport to the next useful record.

A computer program can write a new indexing formula in one line. That does not prove that the underlying records have been physically rearranged in one step. In the paper's machine model, a head moves at most one tape cell per step. Large data movement must therefore be implemented and charged.

Why a calculation can help perform a permutation. For a pair of bits, three XOR updates exchange their values:

$$x \leftarrow x \,\mathrm{XOR}\, y, \quad y \leftarrow y \,\mathrm{XOR}\, x, \quad x \leftarrow x \,\mathrm{XOR}\, y.$$

For initial $(x, y) = (1, 0)$ the states are $(1, 0), (1, 1), (0, 1)$. XOR is addition modulo two: equal bits give zero and different bits give one. Intermediate values are combinations, yet the final result is a pure swap. The large construction also uses intermediate linear combinations to achieve an overall array operation. This two-bit identity only explains the possibility; it does not establish the large construction's speed.

Keep two meanings of geometry separate. The grid here represents positions on an array. The subspaces and ranks used later represent algebraic directions of operators on such arrays. A picture of a short physical path is not by itself a low-rank certificate, and a low-rank identity is not by itself a tape-time bound. The argument needs a precise connection between them.

What to carry forward. The stored order is part of the problem. A useful reordering must be realized by actual steps, not just by renaming indices.

Why moving data is part of the mathematics

Added explanation Read before the machine-model section.

Imagine a long row of labeled boxes, with a head that can move one box per step. The label is a conceptual address; the machine does not get free random access just because we can write $A[i, j]$ on paper. A multidimensional array is flattened into one ordered stream. In lexicographic order, the last coordinate changes fastest. For a 2×3 array the order is

$$(0,0), (0,1), (0,2), (1,0), (1,1), (1,2).$$

Changing to the opposite coordinate order rearranges the records. On this machine, the cost includes getting them into those positions.

A role is a place in a circuit; a row is a block of data. The finite network has W roles, counting both data roles and scratch roles. At a recursion node, complete rows are distributed cyclically among them. A row contains the entire suffix of active address fields. That completeness ensures that each role sees the same local address set. After suitable padding, each role receives exactly a $1/W$ fraction of the logical volume.

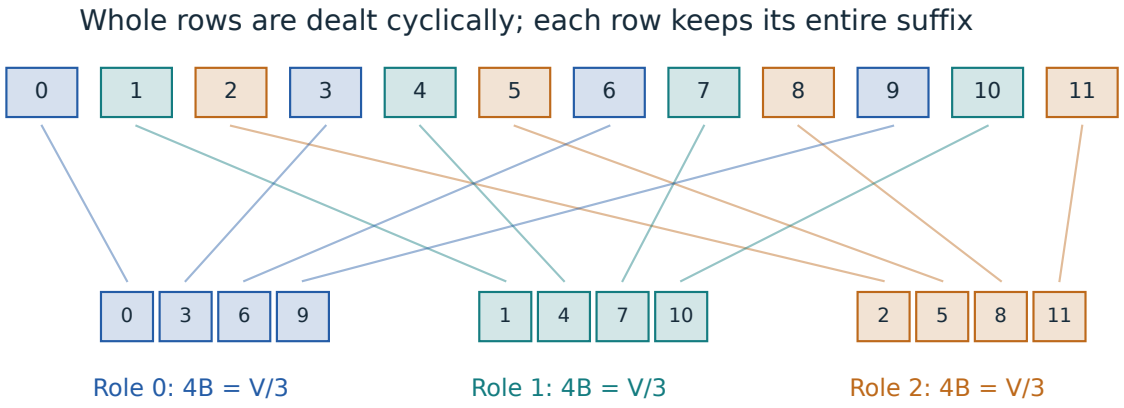


Figure E7. The volume division that drives the recurrence. Each illustrative row has B bits. Twelve rows give $V = 12B$; three role streams each hold $4B$. The actual paper uses much larger fixed networks.

Why auxiliary roles are included. Suppose we counted only useful output rows and forgot workspace. A construction could appear cheap merely by hiding a large amount of temporary data. The network's W counts all roles. Scratch roles can hold arbitrary existing data because the scalar circuit restores them. They are not presumed free zeroed memory. The later framed array operation must also be correct on these roles.

Logical volume is not permission to ignore copying. V names the data volume used to normalize a recursive call. Physical copies, padding, counter updates, tape-head returns, and cleanup still consume time. They are charged as overhead. When a child is active, the paper's schedule parks its parent's streams and restores them on return. The claimed accounting requires that it does not repeatedly sweep through every parked ancestor.

A fixed machine can still process growing descriptors. Input lengths, axis lengths, and call states are stored as data. The number of tapes and the alphabet cannot grow with n. This is why a proof about a finite circuit is only one ingredient: it must be transferred to uniform stream operations with explicit costs for manipulating those descriptors.

What to carry forward. Watch both the number of recursive calls and the amount of data each call processes. A saving in one can disappear if the other is miscounted.

Original source anchors: `02-streams.tex, lem:elementary-streams`.

Vectors, rank, and a frame you can draw

Added explanation The geometry behind the technical vocabulary.

A vector can be as simple as a pair of numbers, such as $(1, 2)$. Draw it as an arrow from the origin to that point. Adding vectors adds their coordinates. Multiplying one by a number scales the arrow. A subspace is a collection closed under these operations: a line through the origin is one example, and the whole plane is another.

A matrix is a rule for combining input coordinates into output coordinates. Its columns tell you where the basic input directions go. If every output lies on a single line, there is only one independent output direction, even if the matrix has many nonzero entries. That number of independent directions is its rank.

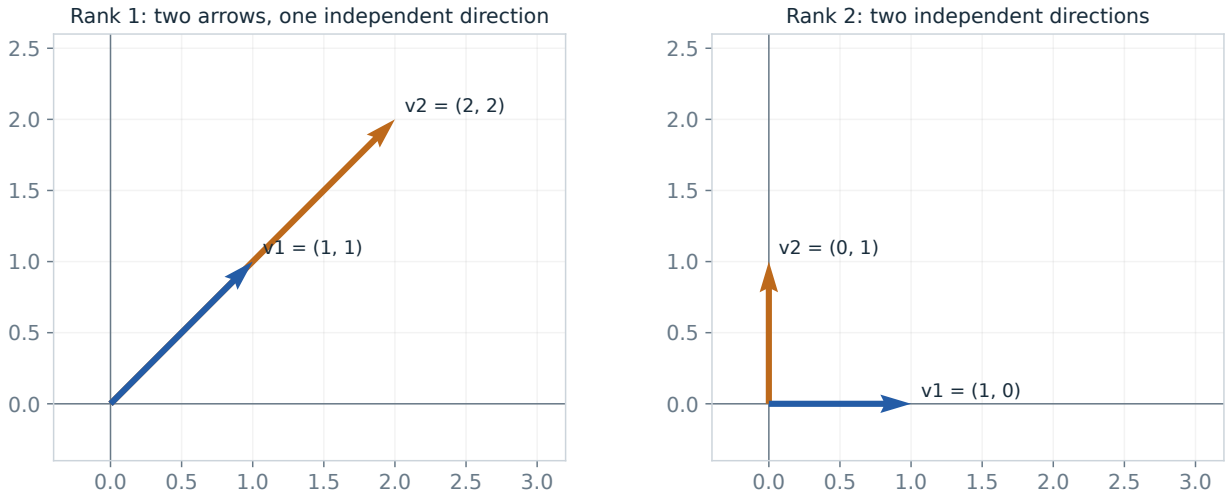


Figure E8. Count independent directions, not arrows. On the left, $(2, 2)$ is twice $(1, 1)$: both arrows lie on one line. On the right, no multiple of $(1, 0)$ reaches $(0, 1)$: both directions are needed. These represent ranks one and two respectively.

The matrices

$$\begin{pmatrix} 1 & 2 \\ 1 & 2 \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

have ranks one and two. The first has four nonzero entries but lower rank. For that first matrix, input (s, t) produces $(s + 2t, s + 2t)$, always on the diagonal line. The second can produce every point in the plane. This is why counting matrix entries would miss the saving sought by the paper.

A frame is a consistent representation rule. Take D to rotate every vector a quarter-turn counterclockwise. The logical value $(1, 0)$ is stored as $(0, 1)$. The logical value $(0, 1)$ is stored as $(-1, 0)$. Adding the stored values gives $(-1, 1)$, exactly the rotation of the logical sum $(1, 1)$. Both inputs used the same rule, so addition remained meaningful.

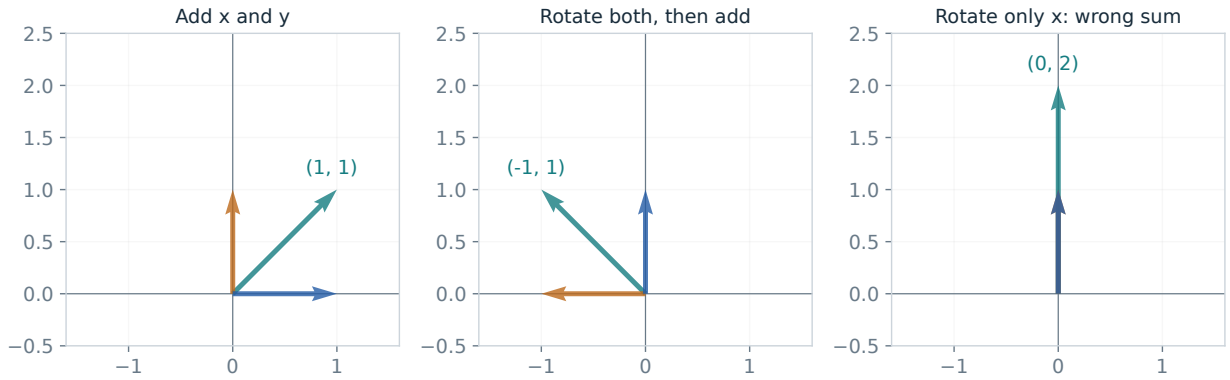


Figure E9. Mixing representations changes the answer. Blue and gold are the two inputs; teal is their sum. Rotating both inputs rotates the correct sum. Rotating just the first produces $(0, 2)$, which is not the intended rotated result $(-1, 1)$. Overlapping arrows in the right panel deliberately show the two equal directions.

In the paper, a frame acts on an entire address-indexed array rather than just a two-number vector. The same law of linearity applies. The later two-by-two tile diagram gives a concrete array version: rotate the address grid of each input and then add matching cells. You get the rotated sum.

Why changing frames can be worthwhile. One representation may make a particular gate cheap while another makes the next gate cheap. The algorithm can switch, provided it pays for switching and maintains the interpretation consistently. Its endpoint frames determine the intended final operation; its intermediate frames are chosen to keep the sum of transition costs small. The actual proof uses more sophisticated operators than rotations, and verifies their residual ranks explicitly.

What to carry forward. Rank measures independent directions. A frame is an invertible representation. Shared frames preserve linear gates; unpriced frame changes would hide real work.

Borrowed workspace and changing representations

Added explanation Read before the finite-network proof.

Two ideas that initially seem unrelated work together here. First, a linear calculation can borrow a register without knowing its contents, provided it removes the influence of those contents and restores them. Second, arrays can be represented in different coordinates at different places in a circuit. The cost of changing representation can be smaller than directly performing the desired overall array operation.

A four-update restoration example. Let a be an arbitrary scratch value. Starting from (x, y, a), perform

$$y \leftarrow y - a, \quad a \leftarrow a + x, \quad y \leftarrow y + a, \quad a \leftarrow a - x.$$

The final state is $(x, y + x, a)$. The initial a cancels from the target, and the scratch register returns exactly to its initial value. This is a toy identity over an additive ring, not a proposed implementation of the full network. In characteristic two, addition and subtraction both become XOR.

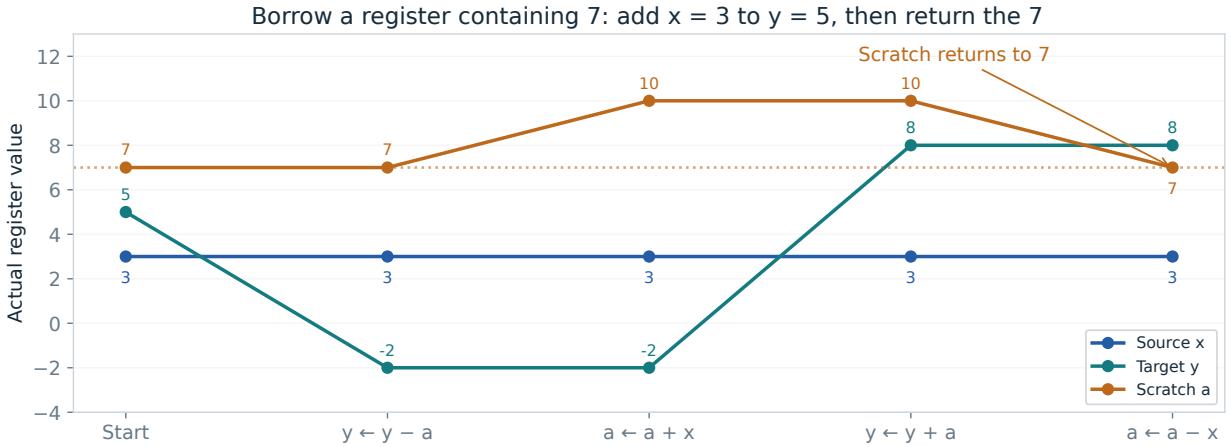


Figure E10. Unknown does not mean unusable. The plotted registers start at $x = 3, y = 5, a = 7$. The target ends at eight, while source and scratch return to three and seven. The symbolic identity above explains why this works for arbitrary initial values, not just this trace.

How the actual network isolates the right source. The original construction indexes values by triples $S, T \subseteq [h]$. Gathering through coordinates shared by the triples gives a coefficient depending on $|S \cap T|$. A side circuit cancels unwanted cross-terms. In the bit version the central coefficient is $|S \cap T|$ modulo two: intersection sizes zero and two already vanish, intersection one is canceled, and the self-intersection three leaves one. Thus the net target update is $y_S \leftarrow y_S + x_S$. For the complex version, the central coefficient is $(|S \cap T| - 1)/2$; side terms cancel the distinct intersection-zero and intersection-two cases. In both cases the scratch-restoration schedule removes the old auxiliary values.

Three bank updates yield

$$(x, y) \to (x, y + x) \to (-y, y + x) \to (-y, x).$$

This is a signed swap; over bits the minus sign disappears. The network alone has not yet accomplished a fast transform. It supplies an algebraic scaffold.

What a frame means. Let g be the logical array on one role. A frame is an invertible linear operator D on its address-indexed entries. The stored array is Dg. A pointwise gate, in contrast, combines different roles at the same address. For a gate adding two roles, a common frame gives

$$Dg_1 + Dg_2 = D(g_1 + g_2).$$

This is ordinary linearity, but it requires the *same* D on every incidence of that gate. With unrelated frames D_1, D_2, the desired identity does not follow.

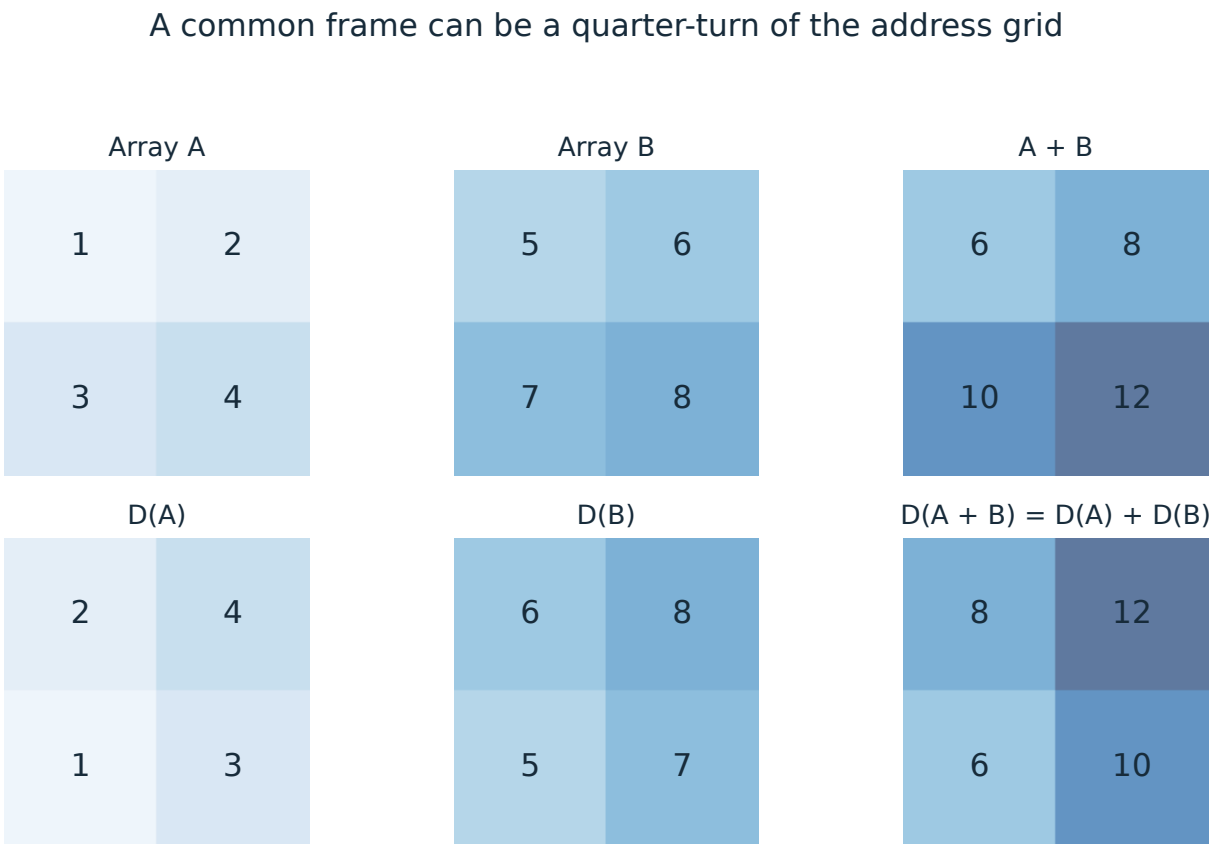


Figure E11. The diagram commutes because the frame is shared. Each panel is an actual two-by-two array. The bottom row rotates the address grid of the top row. Adding the bottom-left arrays gives the bottom-right array, exactly the rotation of the top-row sum.

Between vertices with frames D_u, D_v, the change is $D_v D_u^{-1}$. Following the invariant through the circuit gives the implemented operator

$$D_{\text{out}}\, \mathsf{S}_\Omega\, D_{\text{in}}^{-1},$$

where S_Ω is the scalar circuit applied pointwise. If the scalar circuit is a signed role permutation, each route sees only its endpoint frames and its scalar sign. Intermediate frames cancel algebraically, but their physical changes must still be paid for. Choosing them cheaply is the design problem.

Why the geometry matters. The paper attaches subspaces to vertices. Comparable neighboring subspaces give residual directions whose dimensions count smaller operations. Low rank means fewer independent directions to implement. A matrix with many nonzero entries may still have rank one; for example, $\left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ moves only one independent combination. Conversely, a sparse identity matrix has full rank. The accounting therefore depends on the actual residual operators, their bases, and endpoint contributions, not just the number of visible gates.

The bit circuit combines scalars in $\mathbb{F}_2$ while using rational subspace labels for address operators. The complex circuit combines Gaussian dyadic scalars while using binary subspace labels for phase operators. These are two different uses of a field; confusing scalar values with address labels obscures the construction.

What to carry forward. Restoration makes existing rows usable as workspace. Common frames

turn a scalar swap into an array operation. Residual ranks price the changes between those frames. All three properties are needed.

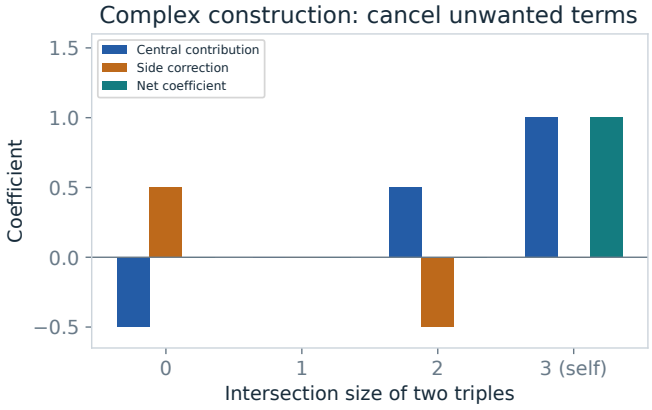
Original source anchors: `03-motifs.tex, lem:scalar-motifs; eq:common-frame-identity`.

How unwanted sources cancel out

Added explanation Read the coefficients, not just the circuit's arrows.

The source circuit temporarily combines many values. The challenge is to leave precisely the source meant for each target. The original construction labels a source and a target with sets of three elements. Their overlap can have size zero, one, two, or three. Two sets of size three overlap in three elements only when they are the same set.

For example, $\{1, 2, 3\}$ and $\{3, 4, 5\}$ overlap once, while $\{1, 2, 3\}$ and $\{2, 3, 4\}$ overlap twice. The central part of the complex construction gives coefficient $(q - 1)/2$ when the overlap is q. That is $-1/2, 0, 1/2, 1$ across the four possibilities. Side contributions cancel the two unwanted nonzero cases. The chart shows both contributions and their sum.



Bit construction: arithmetic modulo two

Overlap	Central	Side XOR	Net
0	0	0	0
1	1	1	0
2	0	0	0
3 (self)	1	0	1

Figure E12. Only the self-contribution survives. The complex coefficients are ordinary signed numbers. The bit construction instead uses modulo-two arithmetic, where $1+1 = 0$; its side contribution cancels the overlap-one case. Both leave net coefficients $0, 0, 0, 1$.

For a target labeled S, every distinct source T therefore has zero net coefficient, and source S has coefficient one. The target receives exactly its intended value even though many other values were used internally. This is stronger than testing one convenient input. It describes every source's contribution to the output.

Where workspace restoration enters. The cancellation table concerns the newly introduced source values. It does not by itself remove arbitrary values that were already in scratch registers. The separate restoration schedule cancels those initial values and returns them. The source proof needs both properties: correct new contribution and unchanged borrowed data. The register-trajectory figure makes the latter property visible for one small example.

What to carry forward. A crowded network can implement a simple map because unwanted coefficients cancel. The proof must account for every contribution, including the values initially occupying its workspace.

Why more calls can still mean less work

Added explanation Count the size of each job as well as the jobs.

Suppose a width-doubling step makes three smaller calls, each on half the data volume. Three calls sounds worse than two, but it does not mean three full-size jobs. Their total visited volume is $3 \cdot (V/2) = 1.5V$. At the next depth there are nine calls on volume $V/4$ each, totaling $2.25V$. The circle areas in the tree encode those shrinking volumes.

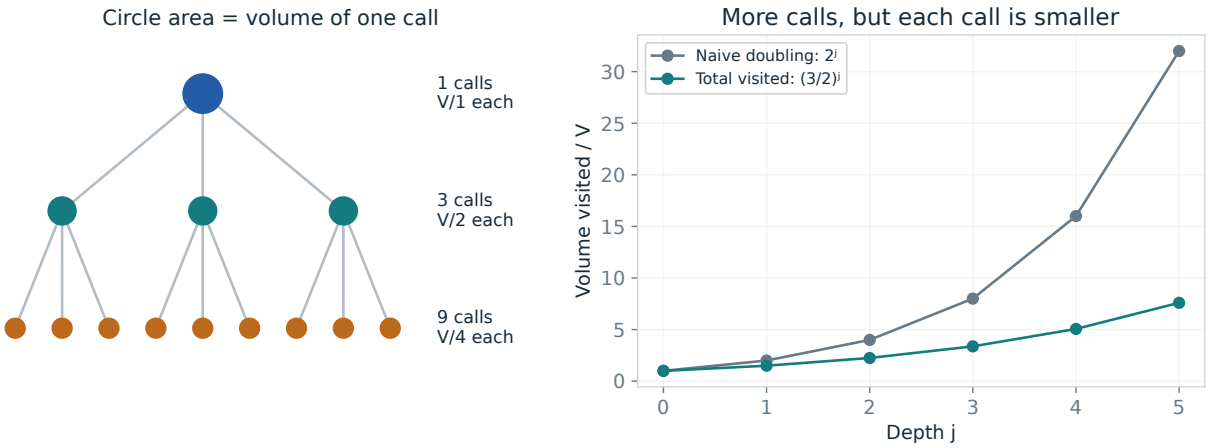


Figure E13. The growth factor is calls times relative volume. This toy tree uses $m = 2$, $W = 2$, $s = 3$. Its first three levels visit V, $1.5V$, and $2.25V$. The right chart compares this with doubling at every level. Calls can run sequentially and reuse storage; the total visited volume describes work, not peak memory.

After j levels, there are 3^j calls and each has volume $V/2^j$. Multiplying gives $V(3/2)^j$. Meanwhile the width factor is 2^j. To express the work as a power of width, solve $2^q = 3/2$. This gives $q = \log_2(3/2)$, about 0.585, below one. The corresponding toy cost grows more slowly than the width itself.

What the real proof must add. The tree charges only the recursive contributions shown. A real algorithm also scans, copies, arranges, and restores records at its nodes. It must bound that overhead, handle widths that do not fit the tree exactly, and keep one fixed machine description. The source reserves strict slack in its recurrence inequality for these tasks. Its numerical saving is much smaller than the one drawn here.

A diagnostic question. What if every one of the three children actually processed the whole volume V? The multiplier would be three, not $3/2$, and this argument would no longer beat a factor of two. This is why the paper's division by W is an implementation obligation rather than a cosmetic normalization.

What to carry forward. A recurrence counts weighted work. The weight is the data volume of each child, including the effects of the required layout and workspace.

How a finite rank saving becomes an exponent

Added explanation Read before address interchange.

Let $C(V, f)$ be a recursive cost for volume V and width parameter f. One schematic recurrence underlying the argument is

$$C(V, mf) \le s\, C(V/W, f) + O(V).$$

The full paper includes layout and descriptor conditions; this display isolates the essential scaling. The parent invokes s smaller operations, each acting on a role stream of volume V/W. Divide by V and write $a(f)$ for a uniform bound on cost per unit volume. Then

$$a(mf) \le (s/W)a(f) + O(1).$$

If $a(f)$ behaves like f^q, its recursive contribution scales by m^q. The critical comparison is therefore

$$\frac{s}{W} < m^q < m, \qquad q < 1.$$

The final proof uses strict inequalities so that overhead can be absorbed.

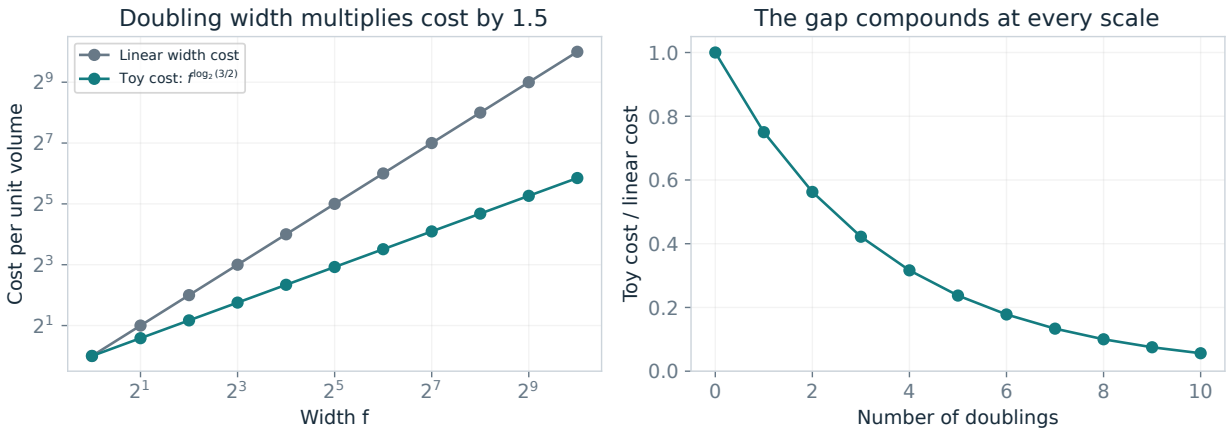


Figure E14. Small children can outweigh a large call count. The plot is an illustrative recurrence with $m = 2$, $W = 2$, $s = 3$; it is not the paper's network and is not a runtime benchmark. Its recursive exponent is $\log_2(3/2) \approx 0.585$.

Why $s < Wm$ is decisive. A width that grows by m would normally cost a factor m. The recursive contribution grows by s/W instead. If $s = Wm$, this comparison gives no strict power saving. If $s < Wm$, an exponent below one can sit strictly above $\log_m(s/W)$. The source constructs two such inequalities, one for address interchange and one for numerical layers. Their rational upper exponents are called τ and σ.

A tiny deficit still gives a positive gap. Define $\eta = (Wm - s)/(Wm)$. Algebra gives

$$\log_m(s/W) = 1 + \frac{\log(1-\eta)}{\log m}.$$

For $0 < \eta < 1$, the logarithm in the numerator is negative. When η is small, the gap is approximately $\eta/\log m$. Approximation helps intuition; the source uses exact rational deficits and strict exponential inequalities to certify a rational exponent. Its original choice is $\tau = \sigma = 1 - 2^{-50}$, not the visibly large saving in the toy plot.

Why a shear is easier to build than a swap. An address shear updates one block by a linear combination of another: $(H, D) \mapsto (H + AD, D)$. The paper factors a rank-a rational matrix A into two invertible triangular factors and a partial permutation with a pivots. Triangular updates follow tape order cheaply. Each pivot needs one smaller interchange. Thus the expensive-call count is the rank a. A suitable fixed odd prime makes the required rational coefficients and inverses valid modulo its powers. Denominators and invertibility cannot be ignored when transferring a rational identity to modular addresses.

The unglamorous conditions are essential. Real field widths need not be powers of m. Row counts may not divide by W. Some layouts contain spectators between active fields. The following source section removes these restrictions and pays for padding and cleanup. The toy recurrence alone is not a proof of those implementation contracts.

What to carry forward. The finite certificate supplies a strict deficit. The tape implementation must preserve its V/W normalization for that deficit to become an exponent saving.

Original source anchors: `03-motifs.tex, eq:explicit-motif-exponents; 04-swap.tex`.

What a transform actually changes

Added explanation Replace a list of values by weights of simple patterns.

A transform is a change in description. Consider the four numbers $[8, 4, 2, 2]$. Instead of listing those values directly, describe them as a sum of four simple patterns. The figure displays each contribution and the result. Add the first point from each of the first four plots: $4 + 3 + 0 + 1 = 8$. At the second point, $4 + 0 + 1 - 1 = 4$. Repeat at the remaining positions to get two and two.

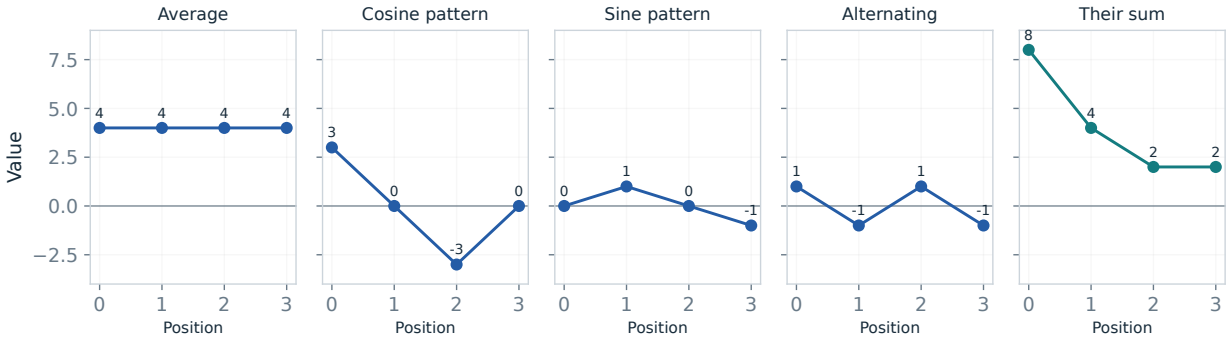


Figure E15. The same four numbers in a pattern description. Average $[4, 4, 4, 4]$, cosine contribution $[3, 0, -3, 0]$, sine contribution $[0, 1, 0, -1]$, and alternating contribution $[1, -1, 1, -1]$ sum exactly to $[8, 4, 2, 2]$. Lines connect four discrete samples; this is not measured sound.

Why call these frequencies? The average does not oscillate. The cosine and sine patterns complete one cycle over the four positions, with different starting phases. The alternating pattern changes sign at every step. In this real-pattern basis, the weights are $4, 3, 1, 1$. These are not the four complex Fourier coefficients in the paper's convention; they are an equivalent elementary way to display the same real data.

Where complex numbers come in. A complex number is a pair of real coordinates written $a + ib$. Addition works coordinate by coordinate. The rule $i^2 = -1$ makes multiplication by i turn the coordinate plane through a quarter-circle: (a, b) becomes $(-b, a)$. This is useful bookkeeping for oscillating patterns; it does not mean the input integers or their final product are imaginary.

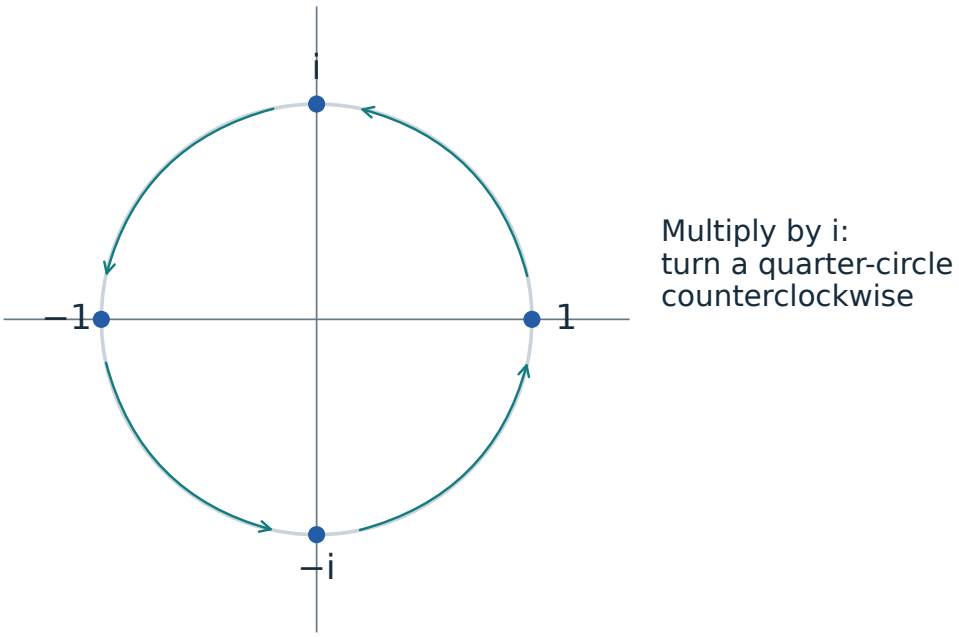


Figure E16. A phase is a position around a circle. Multiplication by i cycles through $1, i, -1, -i, 1$. Each point has magnitude one. The four turns provide the simplest visible example of roots of unity.

For four samples, the ordinary unnormalized Fourier coefficients of $[8, 4, 2, 2]$ are $[16, 6-2i, 4, 6+2i]$ with the negative-exponent convention. Dividing by four gives the normalized version $[4, 1.5 - 0.5i, 1, 1.5 + 0.5i]$. The average is four; the paired complex entries encode the cosine and sine contributions together. You do not need to derive these complex entries to follow the rest of the guide, but the distinction explains why transform scale factors cannot be guessed.

Why this helps multiplication. Shifting a list changes the phase of each frequency in a simple way. Convolution is built from weighted shifts, so in the frequency description it becomes multiplication of matching components, with the chosen normalization accounted for. The hard work is changing descriptions efficiently and reversing the change accurately. The full paper adds carefully chosen layouts and a polynomial representation to address those costs.

What a tensor product means here. First apply a two-entry operation to every horizontal pair in a 2×2 array, then to every vertical pair. Applying the same operation separately on both coordinate directions is its tensor-product action. The terminology becomes compact when there are many axes, but its meaning is still "do this operation in each selected direction while holding the others fixed."

What to carry forward. A transform changes the coordinates used to describe data. Structured patterns can make an operation cheap in those coordinates.

The smallest transform step, applied on many axes

Added explanation Read before simultaneous butterfly layers.

A two-point normalized Fourier transform sends

$$H_0(u, v) = \left(\frac{u+v}{2}, \frac{u-v}{2}\right).$$

The first output records an average; the second records a difference. For $(u, v) = (6, 2)$ the outputs are $(4, 2)$. Recover the original by $u = 4 + 2$, $v = 4 - 2$. The factor $1/2$ matters: this is not the unnormalized Hadamard matrix, nor the unitary matrix with factor $1/\sqrt{2}$.

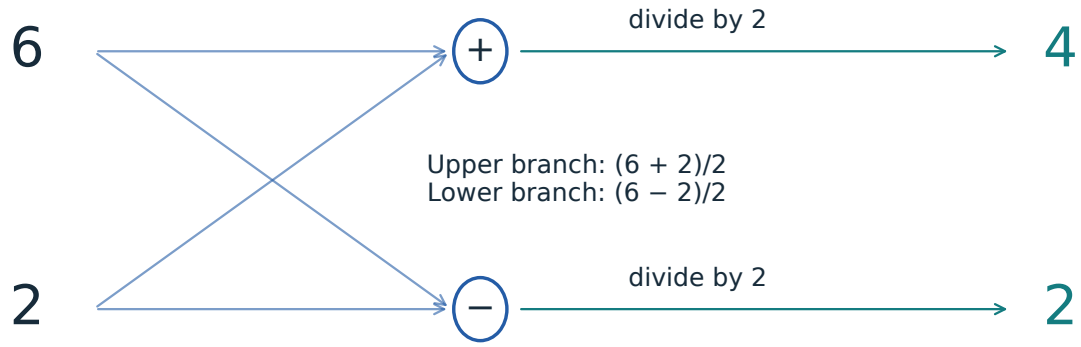


Figure E17. Follow the two arithmetic branches. Both inputs reach both operations. The plus branch returns four and the minus branch returns two after division by two. The entries are a worked example, not intermediate values from the large network.

For a two-axis example, start with the four entries $[8, 4, 2, 2]$ in address order $00, 01, 10, 11$. Applying H_0 to adjacent pairs gives $[6, 2, 2, 0]$. Applying it along the other bit pairs positions zero with two and one with three, giving $[4, 1, 2, 1]$. This is the action of $H_0 \otimes H_0$.

If every input has complex magnitude at most M, each output also does: $|u \pm v|/2 \le (|u| + |v|)/2 \le M$. Thus the completed operation is contractive in the maximum norm. Applying it on several address bits means pairing records that differ in a selected bit, for every setting of the other bits. A naive implementation makes one whole-array pass per selected axis. The proposed layer algorithm organizes those operations jointly.

Why the paper temporarily uses a different kernel. Define X to exchange two entries, $\mathsf{S} = \mathrm{diag}(1, i)$, $a = (1 + i)/2$, $b = (1 - i)/2$, and $C = aI + bX$. The useful identities are

$$C^2 = X, \quad C^4 = I, \qquad H_0 = b\mathsf{S}C\mathsf{S}.$$

The kernel C interacts well with the binary phase frames used to price residual directions. Once its tensor layer is complete, diagonal phases and a known dyadic complex factor produce the desired H_0 layer. These are classical linear identities; the use of phase matrices does not require a quantum computer.

One final rounding, not one rounding per internal gate. The internal values are exact Gaussian dyadics, numbers whose real and imaginary parts are integers divided by powers of two. Internal scratch can grow even though the completed H_0 layer is contractive. The proof therefore needs guard bits for internal magnitudes and denominators. It executes the whole layer exactly and truncates only at its boundary. Contraction alone would not justify discarding bits after every scratch update.

Why selected bits create exceptional addresses. An efficient packed address update may agree with a desired binary basis change on most addresses while carries disturb a small, explicitly described set. The paper isolates that set and repairs it deterministically. The proof needs agreement outside it, invariance of the exceptional set, and a bound on the cost of extracting, sorting, and reinserting full records. "Rare" does not mean random or ignorable: every input must still be correct.

What to carry forward. The second saving batches arithmetic across axes. It must survive basis changes, intermediate growth, and exact correction of exceptional records.

Original source anchors: `05-layers.tex, eq:phase-C; eq:phase-basic-identities`.

Making roots of unity into signed shifts

Added explanation Read before synthetic transforms and ring products.

A transform normally multiplies values by roots of unity. In the synthetic representation, a value is itself a short polynomial, stored as its coefficient list. Work in

$$\mathcal{R}_r = \mathbb{C}[y]/(y^r + 1).$$

This means that y^r is replaced by -1. Multiplication by y shifts the coefficient list and negates the coefficient that wraps around.

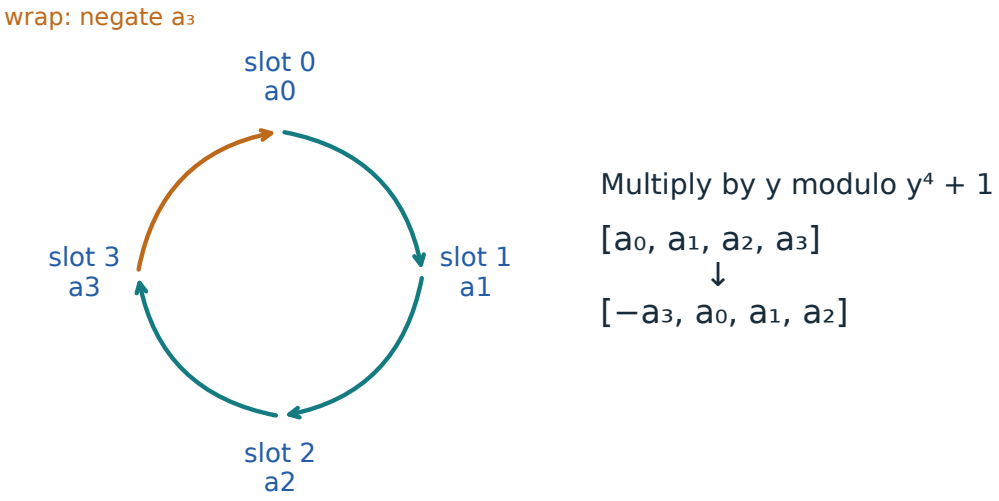


Figure E18. A root operation becomes exact data movement. For $r = 4$, $y(a_0 + a_1 y + a_2 y^2 + a_3 y^3)$ reduces to $-a_3 + a_0 y + a_1 y^2 + a_2 y^3$. No approximate trigonometric multiplication is needed for this operation.

For a power of two t dividing $2r$, the element $\omega_t = y^{2r/t}$ serves as a transform root. Its powers act as signed coefficient permutations. The source proves the needed character cancellation directly. This matters because the quotient ring has zero divisors: dividing by a nonzero-looking ring element is not automatically legal.

Keep the order the algorithm naturally produces. A recursive Fourier transform often produces frequencies in bit-reversed order. For length four, that order is $0, 2, 1, 3$. If both transformed inputs use the same order, corresponding entries can still be multiplied. Sorting each transform into a prettier order would add work without changing the mathematical product. The opposite transform is written to consume the order that is actually present.

Normalization is part of the interface. Both transforms in this manuscript contain a factor $1/t$. Consequently, $F_t^+ F_t^- = I/t$, rather than I. With this convention, for a length-t cyclic convolution,

$$F_t^+\big((F_t^- u) \odot (F_t^- v)\big) = \frac{u * v}{t^2}.$$

One factor of t comes from the forward product normalization, and the other from the normalized opposite transform. Losing either scale would destroy exact recovery. Multidimensional versions replace t by the product of axis lengths, subject to the source's retained permutations.

Why polynomial multiplication does not make the proof circular. Pointwise entries are now polynomials, so their products still need computation. The source packs bounded signed coefficients into larger integers with enough separation to recover each coefficient. It handles centered extraction, signs, and negacyclic wrapping explicitly. For these shorter integers it invokes the already established $O(N \log N)$ multiplier, not the improved algorithm it is trying to establish. The input length N of those calls differs from the original n, and all calls are charged in the final cost table.

What to carry forward. A representation change can replace expensive scalar root multiplications by exact signed shifts, but layout and scaling still cost work.

Original source anchors: `06-transforms.tex, lem:synthetic-characters; lem:signed-ring-product`.

Changing the grid without sorting everything

Added explanation Read before Gaussian resampling.

The natural convolution layout and the efficient synthetic-transform layout do not have the same axis lengths. The multiplication reduction uses pairwise coprime source axes; synthetic transforms prefer powers of two. Gaussian resampling connects them through carefully controlled linear maps. This is an analytic bridge, not ordinary rounding of an index to its nearest neighbor.

In one dimension the source gives an identity of the form

$$P_s F_s = 2^{2\alpha^2} B_0 P_t F_t A.$$

Here A maps from the source grid of length s to a longer grid of length t; B_0 maps back. The P operators are coordinate permutations. The maps are implemented approximately at a stated precision, with explicit error and time bounds. The large known scalar is accounted for in recovery. The inequalities relating s, t, α, p are assumptions of this interface, not decoration that can be dropped when changing parameters.

Why leaving a permutation in a formula can save time. Let P reorder coordinates. If two arrays use the same reordering, then

$$(PA) \odot (PB) = P(A \odot B).$$

Corresponding frequencies are still paired correctly. A consumer that knows the order can use them directly. The simple identity below explains the opportunity; the source proves the more specific identities needed to make all its source and target permutations compatible with the opposite transform.

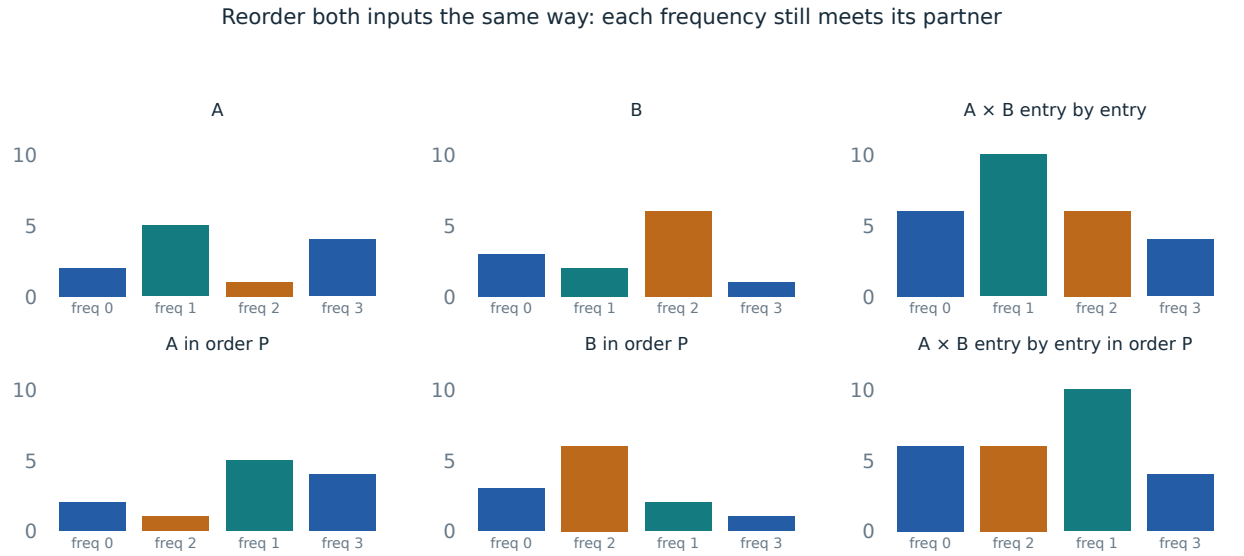


Figure E19. Consistency can remove a needless reorder. The bars show the same permutation on both concrete factors and their pointwise product $[6, 10, 6, 4]$. The diagram alone does not prove that an arbitrary inverse-transform implementation accepts that order. Its interface must establish this.

Where the chirp identity fits. Bluestein's identity rewrites the bilinear exponent using $2jk = j^2 + k^2 - (j-k)^2$. Quadratic phase factors on the inputs and outputs turn a transform into a convolution with a known chirp. The original proof keeps the relevant permutation inside that calculation. A further phase twist reconciles cyclic convolution with the negacyclic polynomial relation $y^r = -1$. Each twist and its removal has both an algebraic and a precision cost.

The error budget has two different kinds of growth. For a composition of exact contractions, existing error is not amplified, so local approximation errors can be added. However, undoing a known large normalization multiplies absolute error. It is therefore wrong to conclude "all transforms are contractive, so the final answer is accurate" without tracking the rescaling. The source chooses sufficient working precision to survive both accumulated local error and the final amplification.

What to carry forward. A layout is part of an algorithm's input and output contract. Preserving a useful order is valuable only when every later step accepts it.

Original source anchors: `07-resampling.tex, lem:no-sort-resampling; 08-assembly.tex`.

Small error must stay small after rescaling

Added explanation A numerical budget you can audit by hand.

Imagine an approximate computation whose current absolute error is at most 0.01. Passing it through an exact contraction does not enlarge that error. If the next approximation adds at most 0.02, the new error bound is at most 0.03. This is a worst-case sum: we do not assume errors cancel or behave randomly.

Now restore a scale factor of eight. The error bound becomes $8 \cdot 0.03 = 0.24$, which is safely below one half. If the scale factor were 32 instead, the bound would become 0.96. That weaker guarantee does not justify rounding to the right integer.

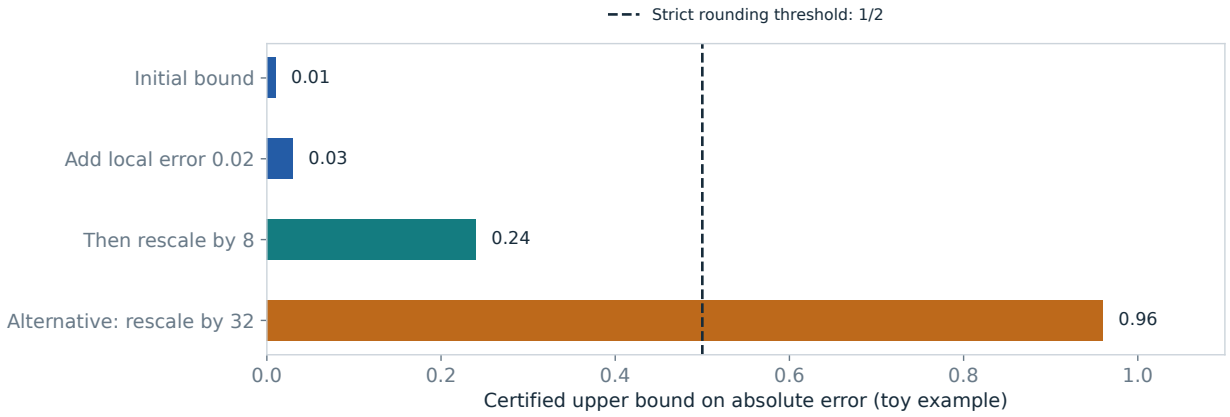


Figure E20. A small internal error is not the final error. The last two bars are alternative rescalings of the same 0.03 bound, not successive operations. Crossing 1/2 means this bound is insufficient to guarantee exact rounding; it does not prove the actual answer is wrong.

The source proof has larger and more elaborate normalizations, but the principle is the same. It budgets local errors, bounds intermediate sizes, and tracks the multiplication of error when scales are undone. It also needs enough bits for intermediate exact dyadic values, even when the completed transform is contractive.

Why strict inequality matters. If the true integer is 12, a result strictly between 11.5 and 12.5 rounds uniquely to 12. At exactly 12.5, the distance to 12 and 13 is the same, so an error allowance of "at most one half" cannot determine which integer was intended. The paper proves a strict final bound for every coefficient and then performs exact carry propagation.

What to carry forward. Approximation is compatible with an exact answer when the fully rescaled error is proved smaller than half the gap between integers.

Why improving one part may change nothing

Added explanation The final saving is limited by the weakest margin.

Suppose three operations have normalized costs $p^{0.80}$, $p^{0.97}$, and $p^{0.90}$. Their margins below a full power of p are 0.20, 0.03, and 0.10. For large p, the term $p^{0.97}$ grows fastest and controls the exponent of the total.

Make the first operation much cheaper, changing $p^{0.80}$ to $p^{0.60}$. Its margin improves to 0.40, yet $p^{0.97}$ still controls the total. Alternatively, improve the second operation to $p^{0.92}$. Now the minimum margin is 0.08, so the combined exponent can improve as well, subject to its remaining overhead and strict-slack requirements.

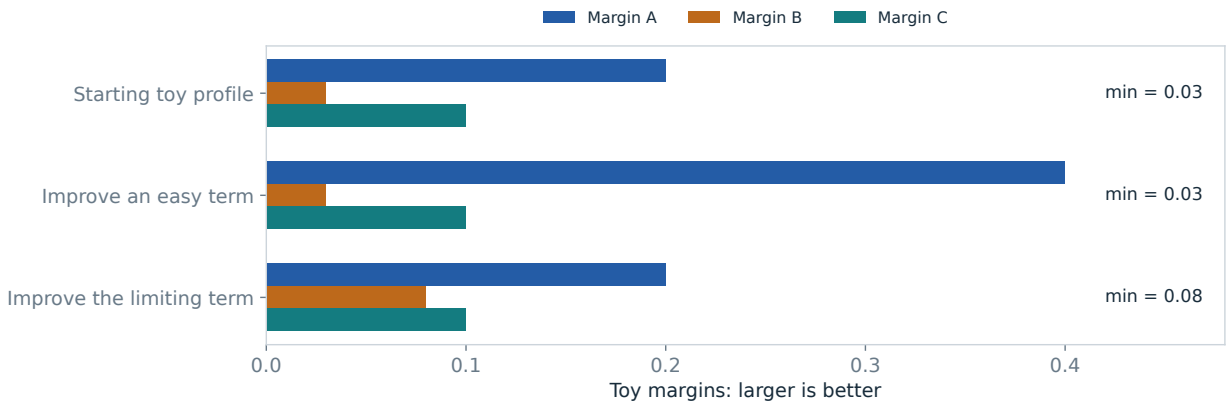


Figure E21. A local gain need not improve the overall exponent. Each row is a separate toy profile, not a historical result. Larger margins are better. The printed minimum stays at 0.03 when an already easy term improves, and rises to 0.08 when the limiting term improves.

The source's final accounting has seven dominant terms instead of three. The figure in the next technical explanation computes their actual original margins. Its logarithmic scale makes extremely small positive values visible. That plot reverses the visual convention: farther right means a smaller margin because the axis is minus the logarithm of the margin. Always read the axis before deciding which bar is better.

Why a little extra room is needed. A cost may also carry factors such as $(\log p)^2$. Such a factor grows more slowly than any fixed positive power p^{ρ}, but it is not a constant. Choosing a final saving strictly below the available margin leaves room to absorb these factors for sufficiently large p. Equality at the boundary would not justify the same argument.

What to carry forward. Track the full cost profile. An improvement is useful at the headline level only when all the limiting terms and interfaces permit it.

Putting exactness and the running time together

Added explanation Read before the final assembly proof.

The paper has two finish lines. It must return the exact product, and every part of the computation must fit the claimed time bound. These obligations are related through precision and storage volume, but neither replaces the other.

Exactness from a strict error interval. The desired convolution coefficients c_j are integers. If the computation produces an approximation $\widetilde{c}_j$ with $|\widetilde{c}_j - c_j| < 1/2$, nearest-integer rounding recovers c_j uniquely. The bound must hold *after* restoring all transform normalizations and digit scales. A tie at exactly $1/2$ is not enough. Once coefficients are recovered, carries and binary output are exact integer operations.

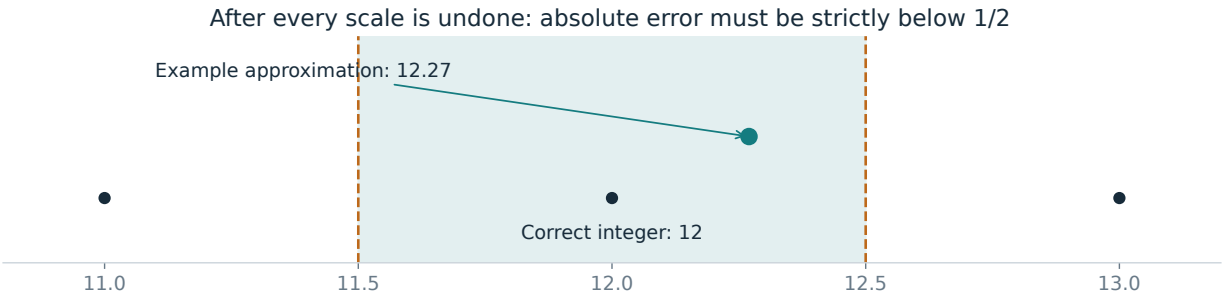


Figure E22. Approximate internal arithmetic, exact final output. The displayed 12.27 is a teaching example. The source's proof supplies a uniform bound on every recovered coefficient; this picture does not measure its actual numerical error.

The source uses radix width $b = \lceil \log_2 n \rceil$ and working precision $p = 6b$ in its original assembly. Roughly, there are n/b digits and $\Theta(p)$ stored bits per coefficient, making the padded volume $V = Tp = \Theta(n)$. This relation is what turns costs per stored bit into a bound in terms of the original input size.

Seven competing bills. The final table separates operations that scale differently with p. Write a normalized term as p^{1-g_i}; then g_i is its margin below a full power of p. With $d = \Theta(p^\epsilon)$, $K = \Theta(p^{\epsilon c})$, and $\ell = \Theta(p^{1-\epsilon})$, the margins are

$$
\begin{aligned}
g_1 &= 1 - \epsilon(1 + c), & g_2 &= \epsilon c(1 - \tau), \\
g_3 &= \epsilon(1 - \lambda'), & g_4 &= 1 - \tau - \epsilon(2 - \tau), \\
g_5 &= \tfrac{1}{4} - \delta - \tfrac{3}{2}\epsilon, & g_6 &= 1 - \delta - \epsilon, \\
g_7 &= \epsilon. &&
\end{aligned}
$$

The total is controlled by the *smallest* margin. Making an already large margin larger may have no effect on the headline exponent.

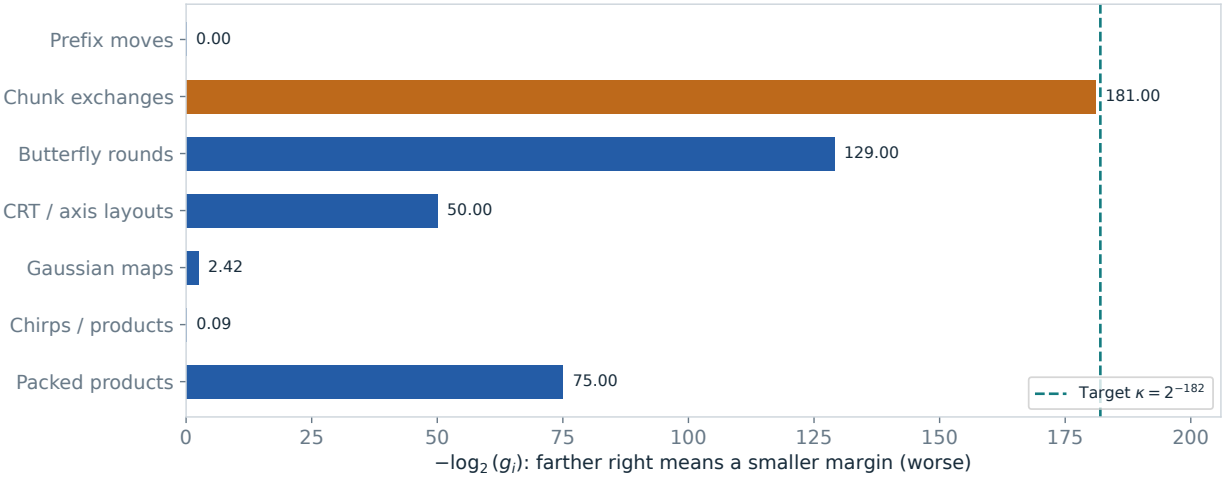


Figure E23. The original parameter choice has a clear bottleneck. These values are computed from exact rational parameters in the original source, not estimated from timings. The horizontal scale is $-\log_2 g_i$: longer bars mean smaller savings. Chunk exchanges have $g_2 = 2^{-181}$; the chosen final saving is $\kappa = 2^{-182}$.

Read one row all the way through. For transform-layout exchanges, there are $O(p/K)$ swaps of width K. Each costs $O(VK^{\tau})$. Thus the total is

$$O(VpK^{\tau-1}) = O\big(Vp^{1-\epsilon c(1-\tau)}\big).$$

This is exactly the second margin g_2. The small quantities multiply: a small width-growth exponent ϵc and a small swap saving $1-\tau$ yield an even smaller final margin. This explains why an impressive local idea may produce a very small asymptotic exponent in the whole algorithm.

Why strict slack is reserved. Let $G = \min_i g_i$. Fixed powers of $\log p$ can be absorbed if $\kappa < G$, since $(\log p)^C = o(p^{G-\kappa})$ for fixed C. The original conservative choice gives $G = 2\kappa$. The final cost then fits

$$O(Vp^{1-\kappa}) = O\big(n(\log n)^{1-\kappa}\big).$$

This argument is eventual. A single fixed cutoff covers all the required inequalities; below it, the machine uses schoolbook multiplication. Enlarging the big-O constant covers the finite set of smaller lengths. Establishing the complete uniform implementation remains part of the proof obligation.

What later community refinements are trying to improve. Some refinements change a finite network or the price of its frame transitions; others reuse workspace, change how stages are combined, or improve an outer parameter choice. These operate at different layers. A new finite profile must still be bound to its actual circuit and then passed through the applicable recurrence and cost analysis. The original seven-margin diagram is a reading aid for this frozen manuscript, not a claim that every later construction uses the same accounting or that it displays the latest community witness.

What to carry forward. Exact recovery needs a strict error bound; the exponent needs a strict margin in every dominant cost. Both must refer to the same algorithm.

Original source anchors: `08-assembly.tex, eq:fixed-parameters; eq:margin-list; eq:final-error`.

What a successful check actually tells us

Added explanation Three claims that should not be confused.

There are different meanings of "it works." A circuit may return the right values on a specified finite set of inputs. A physical program may execute the circuit with a proved resource bound. A theorem may cover every input length, including sizes no experiment can run. Each is valuable, but they are different statements.

Three different claims require three different kinds of evidence

Algebra example	Concrete tape program	All-size proof
Outputs match, and scratch returns to its starting value	Head movement, copies, routing, and cleanup are charged	Every input size satisfies the cost and error bounds
Finite replay checks a concrete instance	A scalar identity alone does not price movement	A finite run alone does not prove every size

Figure E24. Read the claim at the level its evidence supports. The columns separate finite algebra, physical implementation, and all-size analysis. They are obligations to connect, not competing ways of proving the same statement.

A concrete example. The register plot shows that the four updates work when the scratch value starts at seven. The symbolic calculation proves restoration for arbitrary values in its stated ring. Neither statement tells us how much tape movement an enormous array implementation needs. That requires another argument. Similarly, verifying the seven rational margins checks arithmetic within a cost model; it does not establish every algorithmic premise of that model.

Why the guide labels toy examples. Large, visible exponent gaps are useful for learning but would misrepresent the paper if presented as its measured speedup. Small matrices and arrays can demonstrate an invariant without being the actual finite construction. The captions say which numbers come from the source and which are teaching examples. The new figure generators check the displayed arithmetic so that the illustrations explain real identities rather than suggestive pictures.

After this guide. You should be able to describe why convolution helps, why moving data costs time, why common frames preserve addition, how a weighted recursion saves an exponent, and why recovery requires a strict error budget. Following every detail of the source theorem still requires further study of linear algebra, Fourier analysis, and complexity proofs. The expanded edition puts the original arguments immediately after their explanatory material.

What to carry forward. Understanding the architecture is a substantial step. It is not the same claim as having independently verified every part of the theorem.

What follows, and what still needs checking

Added explanation Read before the final corollaries and appendix.

Division and square root. Fast multiplication is a building block for other arithmetic. Newton-style iterations improve reciprocal or square-root approximations at geometrically increasing precisions. If the cost at precision n is $M(n)$ and the smaller stages cost $M(n/2), M(n/4), \ldots$, the sum is controlled by the largest stage for the regular bound in this manuscript. Approximate answers must still be corrected: division verifies quotient and remainder, and integer square root uses exact square comparisons. An approximation routine alone is not an exact division or floor-square-root algorithm.

Matrix transposition. The paper invokes an external reduction from multiplication to rearranging a row-major matrix into column-major order. It accounts for the total number of matrix bits, not just its side length. This is a result about data permutation. It is not a claim about the exponent of matrix multiplication, which is a different problem. The reduction and its input-format conversions retain their own assumptions and costs.

The grouped-rectangle appendix. The appendix extends packed address manipulations to a collection of complete rectangular blocks. "Rectangle" refers to a Cartesian product of address ranges, not a geometric approximation to arbitrary data. Full payloads and destination information must be charged when exceptional records are extracted and repaired. This material supports layout details rather than introducing a separate headline improvement.

A practical reading checklist. When evaluating a proposed improvement, ask these questions in order:

1. What exact scalar operation does the emitted circuit perform, with arbitrary initial scratch values? Does it restore the promised workspace?
2. Do all incidences of each gate have the required common frame? Are all changed operators, signs, source contributions, and endpoints included?
3. Does the paid rank profile describe that circuit, rather than a relaxed or schematic construction? Are finite-field specializations justified?
4. Does the recursive implementation truly give each child volume V/W, with routing, padding, copying, and restoration charged?
5. Do the precision and resampling contracts still apply to the new layouts?
6. Are every final cost margin and the recovery error strictly within their required limits for the same parameter choice?

These questions separate algebra, implementation, and asymptotic analysis. A success at one level is useful evidence at that level; it does not silently discharge the others.

Verification performed for this edition. The frozen original files were checked against their supplied SHA-256 manifest. The digit-convolution example, the four-update restoration identity on 343 integer triples, the two-axis butterfly example, and all seven original rational margins were recomputed. The second edition also checks interpolation, frequency-pattern reconstruction, vector and array frame identities, intersection cancellation, permutation compatibility, and the illustrative error budgets. The figures are generated from those examples or from explicitly labeled schematic identities. Compilation and rendered-page inspection check document integrity and readability. None of these actions constitutes a new audit of the full multiplication theorem.

What to carry forward. The guide explains the proof's architecture and its interfaces. The following source material supplies the detailed arguments; independent validation of the complete theorem is a separate research task.

Notation and source map

Added explanation Use this page while reading the technical sections.

Symbol or term	Meaning in this guide and the original argument
n	Bit length of each input integer.
b	Radix-digit bit width in the original assembly.
p	Working precision; originally $p = 6\lceil \log_2 n \rceil$.
V	Logical stored bit volume at a recursive node.
W	Number of all roles in a fixed finite network, including scratch.
m	Fixed network dimension and recursive width multiplier.
s	Number of paid smaller calls in the recurrence discussion; in resampling, the same letter instead denotes a source length.
d	Number of transform axes, growing slowly with precision.
K	Packed address-chunk width; scales like $p^{\epsilon c}$.
r, ℓ	Polynomial length and its binary logarithm.
T	Padded transform-box size; $Tp = \Theta(n)$. Do not confuse with the time function $T(n)$ or a triple named T.
τ, σ	Rational upper exponents for the bit and complex recurrences.
λ'	Exponent in the completed simultaneous-layer cost after overhead.
κ	Final saving in the exponent of $\log n$.
Role / row	Fixed circuit wire position / complete contiguous suffix block.
Frame	Invertible representation operator on one role's address-indexed array.
Rank deficit	Difference between a full linear call budget Wm and the paid budget s.
Gaussian dyadic	Complex number with power-of-two denominators in both components.
Conditional	The conclusion depends on retained interfaces whose full independent validation is not supplied by a finite certificate.

Where to find each layer of the original source. The filenames below are preserved in the accompanying package. Their theorem labels are more stable references than page numbers after expansion.

Question	Original source file
What machine is counted?	`02-streams.tex`
What does the finite network do?	`03-motifs.tex`
How does rank price movement?	`04-swap.tex`
How are many butterflies batched?	`05-layers.tex`
How are transforms and ring products done?	`06-transforms.tex`
How do grid sizes and permutations fit?	`07-resampling.tex`
Where are exactness and all costs combined?	`08-assembly.tex`
What are the further consequences?	`09-exact-arithmetic.tex`, `10-transposition.tex`

Primary sources and scope references.

1. OpenAI, *Integer multiplication below* $n \log n$, September 23, 2026. Frozen source in the OpenAI math repository.

2. CrocSwap/integer-mult-bounds, community research and attribution. Local scope reference consulted: `docs/audit.md` at commit `87aa555a455cca84a3e3296ced2440623d39fc4d`.

3. David Harvey and Joris van der Hoeven, *Integer multiplication in time* $O(n \log n)$, Annals of Mathematics 193 (2021). The original paper's bibliography and analytic references are retained in the full edition.